

Homework 5

Name: Diya Jain U81673988

1. To prove that the problem complexity of the Balanced Array (BA) problem is $O(n \lg n)$, we can provide an algorithm that solves BA in $O(n \lg n)$ time. One approach is to use a divide-and-conquer strategy to construct the balanced array.

Here's an algorithm to solve BA in $O(n \lg n)$ time:

1. Divide: Split the input array into two halves.
2. Conquer: Recursively solve the BA problem for each half.
3. Combine: Merge the two sorted halves while maintaining the balanced property.

Here's how the combine step works:

- Start with two sorted arrays, left and right, each representing the left and right halves of the input array.
- Initialize two pointers, left_ptr and right_ptr, to the beginning of left and right respectively.
- Initialize an empty result array to store the merged balanced array.
- While both left_ptr and right_ptr are within their respective arrays:
- Compare the elements at left_ptr and right_ptr.
- If the element at left_ptr is smaller or equal, append it to the result array and increment left_ptr.
- If the element at right_ptr is smaller, append it to the result array, but also increment a counter, larger_to_right, which tracks the number of larger elements to the right.
- If the element at right_ptr is equal to the element at left_ptr, append it to the result array and do not increment larger_to_right.
- After processing both arrays, append the remaining elements of the non-empty array to the result array and return the result array.

This algorithm ensures that the result array is sorted and satisfies the balanced property, as the number of larger elements to the right of any element in the merged array can be tracked while merging.

The complexity analysis is as follows:

- Divide: $O(n \lg n)$ divisions.
- Conquer: $O(n \lg n)$ work per division (merging two sorted arrays).
- Combine: $O(n \lg n)$ work to merge two sorted arrays while maintaining the balanced property.

Thus, the overall complexity is $O(n \lg n)$.

2. To prove that the problem complexity of the Balanced Array (BA) problem is $\Omega(n \log n)$, involves showing that any algorithm for BA requires at least $n \log n$ comparisons in the worst case. One way to approach this is by reducing a problem known to require $\Omega(n \log n)$ comparisons to solving BA.

For example, we can reduce the problem of sorting n elements to solving BA. The idea is to transform the input array into a form that BA can handle and then use the BA algorithm to sort the elements.

Here's a high-level overview of the reduction:

1. Take the input array for sorting.
2. Map each element x to a tuple (x, i) , where (i) is the index of (x) in the original array.
3. Solve the BA problem for the mapped array.
4. The resulting balanced array, when sorted based on the first element of each tuple, will give the sorted array.

Consider the array that we want to sort: [5, 2, 8, 1, 4].

- Mapping to Tuples: Map each element (x) to a tuple (x, i) , where (i) is the index of (x) in the original array. After mapping, the array becomes [(5, 0), (2, 1), (8, 2), (1, 3), (4, 4)].
- Solve the BA Problem: Apply the algorithm to solve the BA problem on the mapped array. For simplicity, let's say the resulting balanced array is [(1, 3), (2, 1), (4, 4), (5, 0), (8, 2)].
- Sort Based on First Element: Sort the balanced array based on the first element of each tuple. After sorting, we get [(1, 3), (2, 1), (4, 4), (5, 0), (8, 2)], which corresponds to the sorted array [1, 2, 4, 5, 8].

Since sorting requires $\Omega(n \log n)$ comparisons in the worst case, and the reduction shows that solving BA can be at least as hard as sorting, the problem complexity of BA is $\Omega(n \log n)$.